INF384 · Temas Avanzados en Ingeniería de Software 1

Laboratorio 1 — Punto de partida

Duración: 2 horas · Individual · Este laboratorio es informativo, no lleva nota.

Objetivos:

1. Tener GitHub, GitHub Actions y AWS Learner Lab validados y funcionando.
2. Hacer que un cambio de aplicación se despliegue usando un pipeline.
3. Aprender a leer las cuatro métricas DORA a partir de un repositorio real.

No vamos a desplegar nada en AWS todavía. Hoy AWS es solo para entrar, mirar y entender con qué vamos a trabajar a partir del próximo laboratorio.

Antes de empezar

Necesitas estas cuentas creadas. Asegúrate de llegar al laboratorio con esto creado:

- Cuenta de **GitHub** (con correo PUCP).
- Cuenta de **AWS Academy** ya activada a través del **Learner Lab** (el correo de invitación te llegó; si no entraste todavía, hazlo ahora).
- Cuenta de **Vercel** con el plan Hobby, que es gratis. Puedes crearla con "Continue with GitHub que creaste en el paso 1", es lo más rápido.

Bloque 1 — Accesos y homologación (25 min)

1.1 GitHub con doble factor

Entra a tu cuenta de GitHub y activa la autenticación de dos factores (Settings → Password and authentication → Two-factor authentication).

1.2 GitHub - genera tu token de acceso (PAT)

Entra a https://github.com/settings/tokens y sigue esto exacto:

- Dale a Generate new token → Generate new token (classic). Tiene que ser el classic, no el de fine-grained.
- En Note, ponle un nombre para reconocerlo, por ejemplo inf384-lab.

- En Expiration, elige 1 día. Como GitHub no trae esa opción en la lista, elige Custom y pon la fecha de mañana.
- En los permisos, marca repo (el checkbox de arriba, que cubre todo lo de repositorios).
- Baja y dale a Generate token.
- GitHub te muestra el token una sola vez. Cópialo y pégalo en un bloc de notas mientras dure la clase. Si cierras esa pantalla sin copiarlo, no lo vas a poder volver a ver: te tocará generar otro.
- Ese token es tu contraseña para git de hoy. Cuando en el siguiente paso git te pida usuario y contraseña al clonar o subir código, el usuario es tu nombre de GitHub y la contraseña es el token, no tu contraseña real.

1.3 Haz fork y clona el repositorio en tu cuenta de Github

Este es el proyecto con el que vamos a trabajar. Es el Portal de Estado de Transacciones de PagoSeguro: una sola pantalla donde un comercio afiliado ve el estado de sus transacciones del día.

Primero haz fork del repo base a tu cuenta:

Entra a **https://github.com/jdelcastillos-pucp/pagoseguro-portal-seed-mediocre** y dale al botón Fork (arriba a la derecha). Eso te crea una copia bajo tu usuario: https://github.com/TU-USUARIO/pagoseguro-portal-seed-mediocre.

Ahora clona tu fork, no el repo base. Fíjate que la URL lleve tu usuario:

```
git clone https://github.com/TU-USUARIO/pagoseguro-portal-seed-mediocre.git
cd pagoseguro-portal-seed-mediocre
```

Cuando git te pida credenciales, el usuario es tu nombre de GitHub y la contraseña es el token que generaste, no tu clave.

Date una vuelta por lo que hay dentro. Fíjate en tres archivos, que los vamos a usar todo el rato:

- `app/index.html` — el portal.
- `.github/workflows/deploy.yml` — el pipeline. Ábrelo y léelo aunque todavía no lo entiendas todo.
- `deployments.csv` — un registro de despliegues. Ahorita te explico para qué.

1.4 Fija tu entorno local

Abre el archivo `.nvmrc`. Dice `22`. Eso significa que este proyecto corre con Node 22, y punto. Si usas nvm:

```
nvm install 22

nvm use

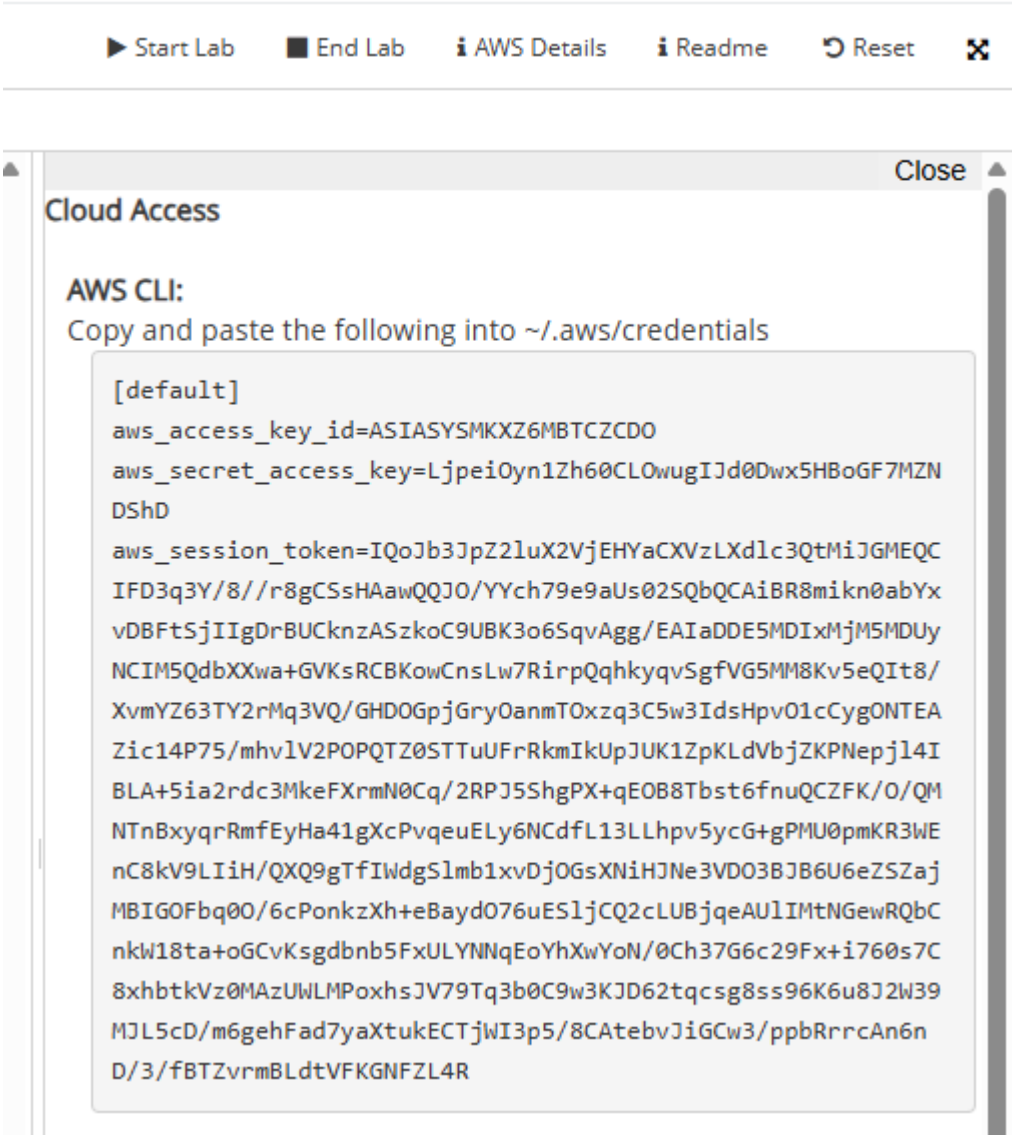
node --version    # tiene que decir v22.x
```

1.5 Entra al AWS Learner Lab

Entra a AWS Academy, abre tu curso y dale al botón **Start Lab**. Vas a ver un panel arriba con un semáforo. Espera a que el punto se ponga **verde**; puede tomar uno o dos minutos.

Cuando esté verde, dale al botón **AWS** de la barra superior. Eso abre la consola de AWS en una pestaña nueva, ya con sesión iniciada. No tienes que poner usuario ni contraseña de AWS: el Learner Lab te la resuelve.

Ojo con esto, porque es importante y lo vamos a retomar: las credenciales que te da el Learner Lab son **temporales**.



La sesión expira. Hoy no nos afecta porque todo lo de AWS lo hacemos dentro de esta misma sesión. Pero anótalo, porque cuando en el Lab 2 intentemos meter esas credenciales en GitHub Actions, esa fecha de expiración va a ser exactamente el problema que tengamos que resolver.

1.6 (Opcional) Token de Vercel

Si quieres ver tu despliegue salir a Vercel al final, necesitas un token. Entra a Vercel → Settings → Tokens → crea uno. Cópialo (solo se muestra una vez) y guárdalo en los secretos de tu repositorio de GitHub: Settings → Secrets and variables → Actions → New repository secret, con el nombre `VERCEL_TOKEN`.

También es necesario agregar las siguientes variables:

- VERCEL_ORG_ID (el valor lo sacas de settings de tu cuenta el valor es team_…)
- VERCEL_PROJECT_ID (el valor lo sacas de settings en tu proyecto el valor es prj_…)

Si no lo configuras, no pasa nada: el pipeline igual va a correr y construir. Solo se salta el último paso.

Bloque 2 — El flujo de git de verdad (25 min)

Hasta ahora clonaste y miraste. Ahora vas a trabajar como se trabaja en un equipo: nadie escribe directo sobre `main`. Todo cambio entra por una rama y se revisa antes de fusionarse.

2.1 Protege la rama main

En tu repositorio de GitHub, ve a Settings → Branches → Add branch protection rule. En el patrón escribe `main` y marca **Require a pull request before merging**. Guarda.

Acabas de prohibirte a ti mismo empujar directo a `main`. Suena absurdo prohibirte cosas en tu propio repo, pero es la regla que en una empresa evita que alguien mande a producción sin que nadie mire. Vamos a vivir con esa regla el resto del laboratorio.

2.2 Haz un cambio en una rama

Crea una rama para un cambio pequeño:

git checkout -b fix/footer-fecha

Abre `app/index.html` y en el pie de página, donde dice `Build: local`, cámbialo por algo que te identifique, por ejemplo tu código PUCP. Guarda y confirma:

git add app/index.html

git commit -m "fix: muestra identificador en el footer"

git push -u origin fix/footer-fecha

2.3 Abre el Pull Request

GitHub te va a mostrar un botón para abrir un Pull Request desde la rama que acabas de subir. Ábrelo. Ponle un título claro. Fíjate que GitHub ya no te deja fusionar de cualquier forma: te obliga a pasar por el PR, porque tú mismo lo configuraste en 2.1.

Antes de fusionar, mira lo que aparece abajo en el PR: GitHub va a arrancar el pipeline automáticamente. Espera al siguiente bloque para fusionar.

Bloque 3 — El pipeline (30 min)

3.1 Lee el workflow

Abre otra vez `.github/workflows/deploy.yml`. Ahora sí, con calma, de arriba a abajo. Quiero que reconozcas la estructura, porque todos los pipelines del curso van a tener esta forma:

- **`on:`** — cuándo se dispara. Acá dice: cuando alguien hace push a `main`.
- **`jobs:`** — el trabajo a ejecutar, en una máquina limpia de Ubuntu.
- **`steps:`** — los pasos, en orden: bajar el código, instalar Node en la versión del `.nvmrc`, instalar dependencias con `npm ci`, construir, y desplegar.

Detente en dos detalles que no son obvios:

Primero, la instalación usa `npm ci`, no `npm install`. La diferencia es que `npm ci` respeta el `package-lock.json` al pie de la letra y falla si algo no cuadra. `npm install` puede actualizar versiones sin avisar. En un pipeline siempre quieres el primero: reproducibilidad sobre comodidad.

Segundo, el paso de deploy a Vercel tiene un `if`. Solo se ejecuta si existe el `VERCEL_TOKEN`. Por eso el pipeline corre para todos aunque no todos hayan configurado Vercel: el que no lo tiene simplemente se salta ese paso.

3.2 Fusiona y mira correr el pipeline

Vuelve a tu Pull Request y dale **Merge**. En cuanto la rama entra a `main`, el push dispara el workflow.

Ve a la pestaña **Actions** de tu repositorio. Vas a ver tu corrida ejecutándose en vivo, paso por paso. Espera a que termine en verde.

Si configuraste el token de Vercel, al final del log vas a encontrar una URL pública. Ábrela: ahí está tu portal, en internet, desplegado sin que tocaras una consola ni corrieras un comando a mano. Alguien hizo merge y el portal se actualizó solo. Eso, que parece poco, es exactamente lo que a PagoSeguro le toma 229 horas hacer según el value stream que levantamos la clase pasada.

Bloque 4 — Leer las métricas DORA (25 min)

Tu repositorio no nació hoy. Trae **40 días de historial de despliegues** ya cargados. Ese historial está en el archivo `deployments.csv`. Ábrelo (en Excel, en el editor, como quieras). Cada fila es un despliegue a producción: cuándo se hizo, qué commit llevó, si pasó o falló, y si falló, cuándo se recuperó.

Un detalle honesto para que no te confundas: este historial **no** lo vas a ver en la pestaña Actions. Los runs de Actions se marcan con la hora real en que corren, y no se pueden inventar hacia atrás. El historial de estos 40 días vive en el `deployments.csv`, que es la forma en que muchos equipos de verdad registran sus despliegues. Puedes cruzarlo con `git log` si quieres comprobar que las fechas de los commits son reales.
Con la información de ese archivo, calcula **cuatro indicadores** del proceso de despliegue de estos 40 días. Si tuvieras que convencer a un jefe de que este equipo despliega bien o mal, ¿qué medirías? Anota los cuatro, con su fórmula y con el número que te da.

Bloque 5 — Rómpelo a propósito (15 min)

Un pipeline en verde da una sensación de tranquilidad que conviene desconfiar. Vamos a ver por qué.

Crea una rama nueva y mete un error a propósito. Abre `package.json` y en el script de `build`, cambia la ruta a un archivo que no existe:

"build": "mkdir -p dist && cp app/NOEXISTE.html dist/ && echo build-ok"

Haz commit del cambio, sube la rama, abre el PR y fusiónalo a `main`. Ve a Actions y mira qué pasa.

La corrida se pone en **rojo**. El build falla porque el archivo no existe. Y acá está lo importante: **producción no cambió**. Si tenías tu URL de Vercel funcionando, sigue mostrando la versión anterior, la buena. El pipeline hizo su trabajo: atajó el error antes de que llegara al usuario.

Cuando termines, revierte el cambio para dejar tu `build` como estaba.

Entregables

Sube a PAIDEIA un documento corto con:

1. URL de repositorio github
2. URL de tu portal desplegado, si configuraste Vercel. Si no, una captura de tu pipeline en verde en la pestaña Actions.
3. Una captura del historial de Actions donde se vea al menos una corrida verde y la corrida roja del Bloque 5.
4. Los cuatro indicadores DORA que calculaste, con su fórmula y su número.
5. Tus respuestas a las tres preguntas de abajo.

Tres preguntas para cerrar

No hay respuesta correcta todavía. Las vamos a retomar más adelante en el curso y quiero comparar lo que piensas hoy con lo que vas a pensar entonces.

1. Con lo que construiste hoy y sin agregar ninguna herramienta nueva, ¿qué tres cosas del proceso de despliegue podrías medir, y de dónde sacarías el dato?

2. El token de Vercel que guardaste en los secretos no expira y permite desplegar lo que sea a producción. Si ese token se filtra, ¿qué puede hacer quien lo tenga, y quién se daría cuenta?

3. En los 40 días de historial del repositorio, el tiempo que tarda un cambio en llegar a producción es de varios días. Casi todo ese tiempo no es el pipeline ejecutándose: es espera. ¿Dónde crees que se va ese tiempo, y en qué se parece esto al value stream que mapeamos en la primera clase?

Antes de levantarte de la máquina

Estás en una máquina que va a usar alguien más. Dos minutos de orden:

1. Cierra sesión de GitHub en el navegador.
2. Borra el bloc de notas donde pegaste el token.
3. Si clonaste el repo en el disco, bórralo. Todo tu trabajo ya está en GitHub, no en esta máquina.